Jpg*Scraper*: An Advanced Carver for JPEG Files

Erkam Uzun, Hüsrev Taha Sencar

***Abstract*—Orphaned file fragment carving is concerned with recovering contents of encoded data in the absence of any coding metadata. Constructing an orphaned file carver requires addressing three challenges: a specialized decoder to interpret partial file data; the ability to discriminate a specific type of encoded data from all other types of data; and comprehensive prior knowledge on possible encoding settings. In this work, we build on the ability to render a partial image contained within a segment of JPEG coded data to introduce a new carving tool that addresses all these challenges. Towards this goal, we first propose a new method that discriminates JPEG file data from among 993 file data types with 97.7% accuracy. We also introduce a method for robustly delimiting entropy coded data segments of JPEG files. This in turn allows us to identify partial JPEG file headers with zero false rejection and 0.1% of false alarm rate. Secondly, we examine a very diverse image set comprising more than 7 million images. This ensures comprehensive coverage of coding parameters used by 3,269 camera models and a wide variety of image editing tools. Further, we assess the potential impact of the developed tool on practice in terms of the amount of new evidence that it can recover. Recovery results on a set of used SD cards purchased online show that our carver is able to recover 24% more image data as compared to existing file carving tools. Evaluations performed on a standard dataset also show that Jpg*Scraper* improves the state-of-the-art significantly in carving JPEG file data.**

***Keywords*—Orphaned file fragment, file carving, JPEG, file type identification**

I. Introduction

Given a fragment of encoded file data with no knowledge on the specifics of coding, unraveling the information contained therein is a challenging task with important implications. The difficulty of this task stems not only from dealing with partial data but also from the fact that encoding may involve many different types of compression as in the case of multimedia data. Orphaned file carving research strives to address this challenge by providing capabilities that are indispensable in a variety of application domains.

In the context of forensic investigations, carving for deleted files and salvaging of data from damaged and faulty media are common procedures. All file systems store file data by splitting data into evenly sized blocks that match the predefined block size of the underlying storage technology, be it random access memory, hard disk drives, solid state drives, or flash memory. By offering the ability to perform data recovery on a block basis, orphaned file carving approaches effectively increase the amount of recoverable evidence from devices. In a manner similar to how data is stored on storage devices, data transmission over network is realized by dividing file data into packet sized blocks. Network monitoring and traffic analysis tools are widely used by operators to get a deeper insight into what type of data is flowing through their networks. In this context, orphaned file carvers provide improved traffic monitoring capabilities by making it possible to sample a few packets from network flows and inspect the content of extracted data.

Essentially, orphaned file fragment carving is the latest addition to existing approaches in file carving where the goal has been to recover data from storage devices without using the file system metadata maintained by the host system to retrieve file data [1]. Early file carvers utilized known file signatures, such as file headers and footers, under the assumption that blocks containing encoded data are allocated in a contiguous manner and attempted to recover files by merging data in consecutive blocks together [2], [3]. In many cases, however, it is not possible to lay out file data blocks contiguously on the storage; therefore, files need to be broken up into a number of fragments. In those cases, this basic approach can only recover the first fragment of a fragmented file, *i.e.*, from the start of the file up to the point at which disruption has occurred, and the rest of file data remains unrecovered.

Later research has led to the emergence of new techniques to enable recovery of linearly or non-linearly allocated fragmented files [4]–[12]. The crux of the fragmented file carving approach involves two steps: first, identification of fragmentation points as file data is being recovered; and second, efficient search for the remaining fragments of a file. Hence, with fragmented file carving, if a fragment of a file is missing, the file can only be recovered up to the missing fragment, and the part of the file data following the missing fragment cannot be recovered. Both of these approaches, in common, rely on the presence of a file header to start recovery as the header marks the start of a file and includes all the encoding parameters needed for decoding. Consequently, if a file is partially intact with its header deleted, both approaches cannot recover any data even though the rest of the file data is intact. Orphaned file fragment carving aims to take file carving capability one step further by removing the assumption that a header is present [13]–[15].

Considering the popularity of photographic media, an orphaned JPEG file fragment carving method is introduced in [14]. Given a block of baseline JPEG coded data, the method recovers the contained partial image content in two main steps. First, it decompresses the partial bitstream by identifying compression parameters such as Huffman codes, chroma subsampling scheme, and quantization tables. Since the space for these parameters can get too large, a brute-force approach by blindly trying all combinations is not viable. To overcome this challenge, a very large set of original

H. T. Sencar is with Qatar Computing Research Institute, HBKU, Qatar and TOBB University, Ankara, Turkey. E. Uzun is with Georgia Institute of Technology, GA, USA (e-mail: hsencar@hbku.edu.qa, euzun@gatech.edu)

photographs captured by close to three thousand camera models are examined to determine commonly deployed encoding parameters. Once entropy decoding is performed, in the second step, partial image content is rendered by determining image width, correctly positioning image blocks, and fixing color anomalies. Test results show that for file fragments of size 8 KiB image content can be extracted with very high accuracy, and even if errors occur, recovered partial images will still deliver visually meaningful information to a practitioner.

A. Limitations of Existing Methods

Conventional file carving techniques, as well as fragmented file carvers, require the presence of JPEG file header; therefore, they lack the ability to recover orphaned file fragments by design [2]–[12]. Further, these methods rely on less accurate methods for discriminating JPEG file data. As a result of this dependency, they are more prone to errors due to misidentification of non-JPEG data as JPEG and due to missing some of the available JPEG data. These translate to longer execution and recovery of lesser amount of data [10], [12], [16], [17].

Prior work on orphaned file fragment carving [14] and subsequent work that applied this capability to the fragment assembly problem [18], [19] implicitly assume that the input to the carver includes encoded JPEG data while considering the use of Huffman code tables introduced in the JPEG standard. Therefore, these works cannot be used in practice without accurate identification of JPEG encoded data and extensive knowledge on utilized Huffman code tables.

B. Our Contributions

In this work, we propose different novel solutions to address these limitations and develop an end-to-end tool, called Jpg*Scraper*[1], to detect encoded JPEG data and carve orphaned JPEG file fragments. We also assess its potential impact on practice of digital forensics. Given a storage volume, Jpg*Scraper* starts by identifying intact image headers to bootstrap its operation. After images corresponding to those headers are recovered, remaining data segments become candidates for further carving. Within these data segments Jpg*Scraper* first identifies and recovers JPEG file fragments with partial headers. Then, it continues to identify and carve out JPEG data fragments for which no metadata information is available.

Our work essentially extends the method introduced in [14], whose details are provided in Sec. II.B, by addressing all issues necessary to build a real-world tool. In this work, we made the following key contributions.

- *Accurate discrimination of JPEG encoded data from all other types of data:* The input to the orphaned file fragment carver must be validated to be JPEG encoded data. An attempt to simply subject every block of data extracted from a typical storage system for recovery would be computationally infeasible. To address this problem, we introduced a new method for identifying JPEG encoded data and quantified its performance on a very diverse dataset. Our method, is able to identify JPEG encoded data at a false rejection rate (FRR) of 0.9% while keeping the false alarm rate (FAR) at 3.65% with error rates computed against more than 198 thousand data chunks sampled from 993 different non-JPEG file types identified based on file extension, the largest and the most diverse dataset used in a study of this nature.
- *Comparison of JPEG encoded data identification methods:* We compared our data type identification method to other techniques including OSCAR [16], Reduce [12], and Extreme Learning Machine (ELM) [10] on the DFRWS-2007 challenge dataset [20] in their ability to correctly identify JPEG file data. Results show that our proposed method outperforms existing methods with a discrimination accuracy of 99.08%.
- *Reliable identification of partial JPEG headers:* JPEG file format allows storing plenty of metadata information as part of the file header (*i.e.,* the ExIF header) even including a thumbnail image in JPEG format. Therefore in a given block of JPEG file data, entropy coded data might be preceded by parts of JPEG file header which will hinder the operation of the carver. To ensure successful carving, we present a method to reliably identify partial header data, and thereby the starting point of entropy coded data in the input block. Our evaluation on a large image set revealed that the method can detect partial file headers of baseline JPEGs with zero false rejection rate (FRR) while keeping the false alarm rate (FAR) at 0.1% for non-JPEG file data.
- *Robust carving capability under diverse practical settings:* JPEG image coding involves several parameters that are relevant to carving, such as the Huffman code tables used during entropy coding. Although these parameters can be customized for each image, our earlier findings revealed that camera makers primarily utilize precomputed values. Therefore, prior knowledge on these parameters are of vital importance for the success of carving. To ensure a comprehensive coverage of parameters, we examined images captured by 3,269 camera models and edited by several tools and incorporated our findings into the tool.
- *Reliable error detection mechanism on partial bitstream decoding:* We proposed a new error detection mechanism to detect an incorrect encoding setting in a very fast manner when decoding a partial bitstream. Our evaluations show that we can determine a mismatch in the assumed Huffman code tables within the first 1 KiB of data at very low error rates (FRR=0.29%, FAR=0.35%).
- *Measurement of the forensic utility:* Proposed carver obtains a new class of evidence; therefore, its true impact on practice depends on the amount of additional evidence it can retrieve. Based on results obtained on a set of used SD cards acquired from online sellers, Adroit Photo Forensics (APF) tool (a state-of-the-art file carver developed for fragmented files [21]) recovered a total 9.83 GiB of JPEG data with complete file headers. Jpg*Scraper* recovered an additional 2.4 GiB of JPEG data from the same dataset by carving out files that have partial headers (0.7 GiB) and orphaned file fragments (1.7 GiB). This finding has

[1] https://github.com/euzun/jpgscraper

important implications on digital forensic investigations and, in a complementary manner, on user privacy due to improper data purging practices.

- *Comparison with existing recovery tools:* We compared recovery performance of Jpg*Scraper* with state-of-the-art JPEG file carvers like APF, JPGCarve [12], PattrecCarv [22] and rx_myKarve [10] on the DFRWS-2007 dataset as it is commonly used by these works. In our comparison, we measured additional JPEG file data recovered by our tool on top of each method. We note that JPGCarve, PattrecCarv, and rx_myKarve consider only recovering multi-fragmented, thumbnails and embedded, and linearly fragmented JPEG files, respectively. Considering the recovery context of each method, we recovered the most additional fragments on top of rx_myKarve and PattrecCarv with around 60% of all JPEG fragments. On the other hand, we recovered the least additional data (0.12% of all JPEG files) with respect to APF. As compared to recovery results on SD cards, the improvement here is marginal because DFRWS-2007 dataset does not intentionally include orphaned file fragments.

In the next section, we briefly describe the main steps of the proposed orphaned JPEG file fragment carving method and specify details of the workflow for how this method will be used in practice. Our proposed solutions concerning how the above cited issues need to be dealt with are addressed in Sect. III. The recovery results on a set of used memory cards bought on Ebay are presented in Sec. IV to better judge the operational value of the method. Comparison results with existing methods are presented in Sec. V. Our conclusions are given in Sec. VI.

II. Revisiting Orphaned JPEG File Fragment Carver

The problem of developing an orphaned file carver can be abstracted as reconstructing a generic header for a given block of data to bootstrap a custom decoder. Hence, one aspect of the problem reduces to determining all the parameters and settings that are necessary for successful decoding. In the context of JPEG encoding, these involve the compression mode, chroma subsampling scheme, Huffman code tables, quantization tables, and the image width at the least. Some of this information, such as Huffman codes and the chroma subsampling mode cannot be inferred from analysis of the encoded data at hand. At the same time, however, the combined space for all these parameters is too large to adopt a brute force solution. Since JPEG images are mostly created by digital cameras, one way to approach this problem is by examining the headers of images captured by a variety of cameras to determine commonly used parameters and settings. Once obtained, this information can later be used for brute forcing these parameters in order of likelihood of being observed in images.

A. JPEG Encoding Settings

We examined around 10 million publicly shared user photos on Flickr photo sharing website to obtain statistics on JPEG compression parameters. Flickr API provides access to image metadata at the elemental level, and the full ExIF headers can only be obtained by downloading each photo. To obtain reliable information, we subjected these images to elimination based on four factors [14], [23]. Accordingly, images in JPEG format, tagged as *original*[2] with resolution higher than 1024×1024 pixels were retained and those with no or very sparse metadata and less than three color channels were disregarded. These restrictions overall yielded $7,989,447$ image entries. We were able to download $7,095,502$ of these images. After extracting detailed ExIF information of downloaded images by using JPEGSnoop tool [24], we found that they are captured by 3,269 different camera models manufactured by 190 different camera makers including all of those in the market today. We then examined information stripped from ExIF headers of all images to obtain statistics on encoding parameters (*e.g.*, Huffman code tables, quantization tables, chroma subsampling structures, and image dimensions). Unlike [14], we kept post-processed images by known image editing tools since the recovery tool will be exposed to all types of images in practice. We refer to this dataset as *Flickr* dataset in the rest of the paper. Next, we briefly summarize our findings on the collected data in comparison to our previous measurements on a smaller set of images.

Huffman Code Tables: In our analysis, we found that 99.96% of the images were encoded in JPEG baseline mode. This is in agreement with our earlier measurement (99.51%) and shows that the proposed method will be very effective in practice. In baseline mode, a set of $4-6$ Huffman code tables are used for encoding DC and AC coefficients of luminance and chrominance color components of an image depending on whether the two chrominance components coded using the same codes [25]. We observed $640,808$ distinct sets of Huffman tables in our dataset. Out of these, $635,288$ (99.14%) sets, which comprise 8.95% of images in our Flickr dataset, were unique, indicating that they were custom generated per image at the time of coding. We observed that the recommended set of Huffman tables by the JPEG standard is used in coding of 76.94% of the images. Although this shows a significant reduction from the earlier finding of 98.61% among original camera photos, it is still interesting to note that even photo editing tools do not optimize tables for each image. The other most frequently deployed Huffman code table sets are used in coding of 6.41%, 5.1%, 1.57%, 0.55%, 0.28% and 0.12% of the images. Since each of the rest of the set of Huffman tables were utilized by very few images, we consider the top seven sets as the most frequent ones. We incorporate these findings into our carving method as built-in parameters and try each set of tables one by one in order of decreasing frequency during decoding partial bitstream.

Chroma Subsampling Schemes: We encountered all four chroma subsampling schemes, namely, the horizontal, vertical, horizontal and vertical and no-sampling. Similar to our previous analysis, horizontal sampling scheme is the most frequently used one (by 50% of images) out of the four schemes. Most notably, no-sampling scheme is observed to be used by 18% of images. This indicates a significant increase

[2]This refers to the version of photo uploaded initially by the user, prior to re-encoding by Flickr

from our earlier measurement of 1%.

Image Dimensions: Resolutions of images in our data sets varied from one mega-pixel to 404.51 mega-pixels. We determined $81,905$ distinct image resolutions of which 60,844 (74.29%) were seen only once. Given only about 0.1% of the total number of images have unique dimensions, we can assert that cropping or scaling of images is not very frequently practiced. However, we observed 20 times more distinct dimensions than our previous analysis. This is most probably due to custom dimensions used by editing tools.

Quantization Tables and Quality Factors: A quantization table (QT) is an 8×8 matrix of integers specifying the quanta values used for quantizing DCT coefficients in each of 64 frequency bands. In the baseline mode of JPEG coding, typically two quantization tables are used, one for luminance and the other for chrominance components. Quality factor (QF) is a term used to denote the strength of compression. QF values vary between 1 and 100 and are used to control the generation of a scaled versions of the QT during encoding for better compression. In this work, we observed $18,372$ distinct sets of QTs in which 53% of them are used only once. Similar with our previous observations, above 90% of images are compressed with a QF of $80-90$. Hence, carving can be performed using a few default quantization tables without a significant impact on the quality of the recovered file fragment. Moreover, only 40 sets of QTs are used by 75% of images in the Flickr dataset. Therefore, we simply used the most frequently appearing QT for each chroma subsampling scheme while rendering recovered fragments.

B. Orphaned Fragment Carving Steps

Orphaned file fragment carving is much more complicated than just reconstructing a header as it also requires a specialized decoder that can interpret a partial bitstream. In a JPEG coded bitstream, codewords of different Huffman codes are interleaved and mixed with coefficient values. Further, JPEG encoder performs byte stuffing to distinguish between encoded data and marker codes which are meant to be ignored during decoding. The main challenge of carving, however, stems from the fact that encoding has to start at some arbitrary bit location in the compressed bitstream, which may not correspond to a (Huffman) codeword boundary. In dealing with these issues, our method relies on self-synchronization property of Huffman codes and the composition of 8×8 coefficient blocks. Due to the self-synchronization property, Huffman codes are resilient to errors like insertion, deletion or substitution of bits. Our results show that the synchronization phenomenon also applies to JPEG encoded stream where Huffman codes are interleaved and mixed together with coefficient values. We incorporate this finding along with the frequent usage of end of block (EOB) codewords (94.11% of coded 8×8 blocks [14]) in order to determine in which order Huffman tables must be used for decoding.

Equipped with this information for a given fragment of a JPEG baseline image coded using precomputed Huffman code tables, our carving method follows the process flow chart presented in Fig. 1. Below we briefly explain the involved

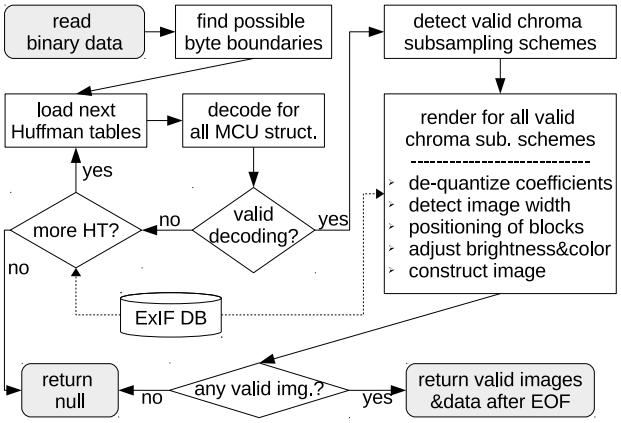


Fig. 1. Process flow diagram of orphaned JPEG file fragment carving in practice. We used this system as a black-box component in Jpg*Scraper*. (MCU stands for minimum coded unit, HT refers to Huffman code tables, and EOF stands for end of fragment.)

recovery steps. (Further details on these processing steps can be found in [14].)

Decoding the partial bitstream: At first, byte boundary in the bitstream is detected to identify and discard (non-encoded) bytes stuffed at the encoder. This is realized by exploiting the fact that JPEG standard disallows certain byte patterns to appear in the byte aligned code. This is followed by determining the employed subsampling scheme. For this, we decode the orphaned JPEG file fragment assuming all possible sampling structures which are also called as minimum coded unit (MCU) structures. If we observe a number of error conditions related to the use of incorrect set of Huffman tables or incorrect MCU structure, we terminate the decoding process. When no errors are encountered, we further verify that decoded data exhibit the characteristics of quantized discrete cosine transform (DCT) coefficients, and proceed with rendering steps.

If all attempts fail, a new set of Huffman code tables is selected and this step is repeated until decoding succeeds. Our results show that, an overall accuracy of 96.75% can be achieved in detecting correct subsampling scheme even for 4 KiB JPEG data segments.

De-quantization: Once entropy coded DCT coefficients are decoded, they are organized into 8×8 blocks, de-quantized using the quantization table that was encountered most frequently for the corresponding subsampling structure, and inverse transformed to spatial domain.

Detecting image width: Recovered image blocks are analyzed by measuring block-level discontinuities between subsequent blocks. Since there will be strong correlations between spatially adjacent blocks, periodic disruptions in the observed pattern will indicate the start of a new image row. To detect the image width reliably, correlation and clustering based measures are used. Our measures achieved an overall accuracy of 95.13% in determining the correct image width considering 64 KiB data segments.

Positioning image blocks: Since image data is only par-

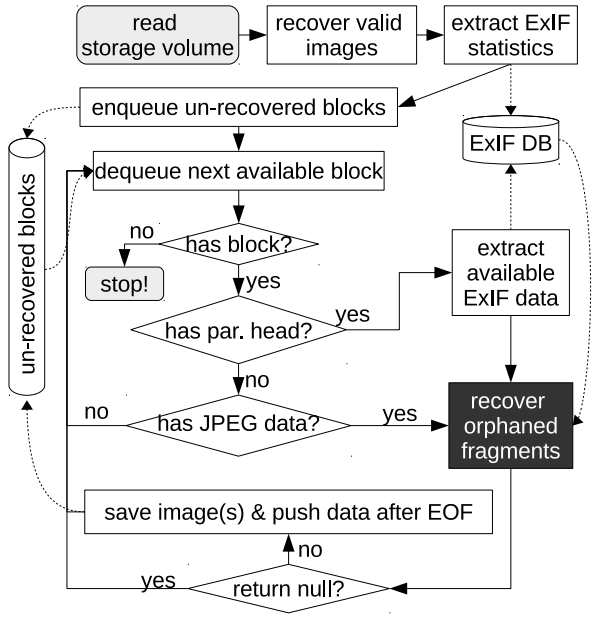


Fig. 2. Process flow diagram of the orphaned file fragment carver.

tially available, the actual horizontal position of the recovered image blocks in the image cannot be known. However, when image width is correctly detected, incorrect positioning of the recovered blocks will only cause a circular shift in the image content (later pixels are followed by initial pixels at each row). By measuring intensity variation along columns of blocks, this offset can be restored. In our previous analysis, we achieved an accuracy of 91.79% to correctly restore the correct offsets.

Brightness and color adjustment: Since JPEG encodes DC coefficients differentially with respect to the preceding blocks, a decoded DC coefficient value of an intermediate block will have a difference value rather than its actual value. To compensate for the missing base value, we perform histogram stretching on the resulting image and shift the pixel values to fill the entire range of values.

Validation: The resulting image fragment is finally assessed for whether it exhibits the natural image statistics before it is considered successfully recovered. To eliminate incorrectly recovered images, regularities in multi-scale representation of images are utilized. This is realized by computing first- and higher- order statistics from sub-band coefficients and using them as features to build a classification model [26].

Overall, the above pipeline decodes a 4 KiB of JPEG data segment in 8.7 milliseconds and renders corresponding RGB image in 65.1 milliseconds with our single threaded and CPU-based C# implementation running on a commodity laptop with Intel i7-6500U CPU @ 2.5GHz processor, 8 GB memory, and 64-bit Windows-10 OS. It should be noted that, GPU-based or multi-threaded implementation would fasten this process to render orphaned fragments at real-time. We refer the reader to [14] for further details on each step. Jpg*Scraper* builds on this basic capability to construct a real-world carver.

III. Carving Challenges

Given a forensic image of a storage device, our carving operation follows the process flow diagram shown in Fig. 2. As the first step, all images with a valid header are recovered, and remaining data segments are retained for further analysis. (This step can be realized by using one of the conventional file carving tools or by Jpg*Scraper* itself.) Here we note that image files stored together are more likely to be interrelated as they might have been captured by the same camera, edited by the same editing tools, or downloaded from the same Web pages. To exploit this spatial locality, coding parameters obtained from those recovered images are incorporated into parameters obtained from the Flickr dataset and are given higher precedence. This overall saves recovery time as Huffman codes and the color sampling scheme need not be searched, and further, it yields better quality images due to knowledge of the image width and quantization parameters.

This first step is followed by checking for the presence of partial ExIF metadata at the beginning of remaining data segments to identify runs of entropy coded data. We also determine if any of the necessary coding parameters can be recovered from the partial header to leverage orphaned file fragment carving. If no traces of ExIF data is found, the type of data in the unrecovered segment is validated to be in JPEG encoding format to ensure non-JPEG data is not subjected to carving.

JPEG encoded data segments are then recovered by initializing the orphaned file carver from a dictionary of parameters that involve coding parameters such as Huffman code table sets, chroma sub-sampling schemes, image width and quantization tables, sorted based on their occurrence frequency among the recovered data and the Flickr dataset. The orphaned file carver, described in Section II-B, is integrated here as a black-box capability ('recover orphaned fragments' processing block) as shown in Fig. 2.

For each singled out unrecovered data segment, this step is executed until successful recovery by changing parameters in each run. The carver also returns residual, unrecovered data chunks when a fragmentation point is encountered within the data while decoding the partial bitstream. Such residual data is pushed at the top of the unrecovered data segment queue so that it could be reprocessed on the next trial. These steps are repeated for all unrecovered data segments.

Overall, development of an orphaned JPEG fragment carver requires addressing a number of challenges described in the above work-flow. These include detecting partial ExIF metadata, identifying JPEG encoded data, effectively determining the correct set of Huffman tables, and detecting the end of a fragment. In the following, we describe how we address these challenges and introduce additional datasets we used for parameter tuning and validation.

A. Training Datasets

These datasets include a large collection of JPEG and non-JPEG file data and are used to test the effectiveness of proposed approaches in identifying partial ExIF headers, discriminating

JPEG file data from non-JPEG file data, and to identify correct Huffman code tables.

Non-JPEG File Dataset: To create a comprehensive set of non-JPEG files, we searched for all file types that could exist in a commodity computer with Windows 10 OS and gathered more than 266 thousand files. These files are then filtered in a manner similar to that described in [16]. More specifically, we discarded all files with size less than 4 KiB and all file types that have less than 10 file samples. In doing this, we considered each file extension as an indicator for a different file type, and files without an extension are not considered. Then, for each file type, we randomly selected 100 samples (if there are less than 100 files with the same extension, we selected all that is available to represent that file type) and randomly sampled up to 10 data chunks of size 4 KiB in a non-overlapping manner depending on the size of the file. This provided us with 19,885 different files and 198,827 non-JPEG encoded data chunks from 993 file types.

JPEG File Dataset: To create a dataset equal in size with the non-JPEG dataset, we randomly acquired 4 KiB of JPEG encoded data chunks from 200 thousand distinct images in the Flickr dataset.

7HT Dataset: This dataset is created to test how effectively Jpg*Scraper* is able to identify the correct set of Huffman code tables when decoding data. It contains 7 groups of 100 data chunks, randomly sampled from files entropy coded with the seven most frequently used Huffman code table sets observed in Flickr data. The size of data chunks is set to 1 KiB, instead of 4 KiB, which is found to be sufficient to identify the correct set of Huffman code tables.

B. Identifying Partial ExIF Header

Conventional file carving tools skip recovery of JPEG files without a complete ExIF header. However, partial headers may still provide some of the necessary decoding parameters. In our case, these coding parameters are added to the dictionary of parameters and used with higher precedence during recovery from the forensic image at hand. At the least, a partial header could reveal the start of scan (SOS) point which marks the start of the encoded data. This alleviates the synchronization burden as well as providing the carver with the initial DC values of the DCT coefficients. Since subsequent DC coefficients are incrementally coded with respect to this initial value, it enables rendering of the partial image with more precise brightness and color values. Overall, this saves execution time and yields better recovery.

In JPEG standard, the SOS point is initialized by a 16-bit SOS marker (0xFFDA) [27]. However, the SOS marker could not be solely relied on for detection of the SOS point since this short bit pattern (0xFFDA) could also be randomly encountered within all types of data. In our training datasets, we encountered at least one 0xFFDA keyword within, respectively, 20% and 10.1% of JPEG and non-JPEG encoded data chunks. On the other hand, it is possible to identify SOS point by considering longer combination of parameters as the keyword. According to the scan header syntax presented in Fig. 3, it is possible to construct 10-14 bytes long keywords for detecting SOS point in JPEG baseline-coded data [27]. The definition, size, and possible values for each parameter shown in Fig. 3 are presented in Table I. Hence, many combinations of keywords are possible, especially for the progressive and lossless compression modes of JPEG. In practice, however, many of these combinations are unlikely as majority of JPEG images (99.96% in Flickr dataset) are encoded in baseline mode.

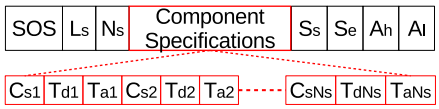


Fig. 3. Scan Header Syntax

TABLE I. DEFINITIONS AND POSSIBLE VALUES OF SCAN HEADER PARAMETERS SHOWN IN FIG. 3

Parameter	Definition	Size(bits)	Values
SOS	Start of scan marker	16	0xFFDA
L_s	Scan header length	16	$6 + 2 \times N_s$
N_s	Number of image components	8	1-4
C_{s_j}	Scan component selector	8	0-255
T_{d_j}	DC table destination selector	4	0-1
T_{a_j}	AC table destination selector	4	0-1
S_s	Start of spectral selection	8	0
S_e	End of spectral selection	8	63
A_h	Succes. approx. bit pos. high	4	0
A_l	Succes. approx. bit pos. low	4	0

To determine most likely keyword patterns associated with an SOS header, we examined headers of 500K images in the Flickr dataset to identify all combinations used as part of the scan header of JPEG baseline images. Table II presents five different keyword patterns that are encountered at the SOS point in the bitstream. Essentially, we observed that second and third scan header keywords (SH_2 and SH_3) are the most frequent ones with appearance in 99.71% and 0.26% of images.

TABLE II. OCCURRENCE RATES OF 0XFFDA MARKER AND DIFFERENT SCAN HEADERS ENCOUNTERED IN JPEG BASELINE IMAGES WITHIN BOTH JPEG AND NON-JPEG ENCODED DATA CHUNKS

Marker	Header Hex	in JPEG(%)	in non-JPEG(%)
SOS	0xFFDA	19.99	10.90
SH_1	0xFFDA0008010100003F00	0	45e-4
SH_2	0xFFDA000C030100021103110003F00	0	94e-3
SH_3	0xFFDA000C030000011021100 3F00	0	0
SH_4	0xFFDA000C030000010002000 3F00	0	5e-4
SH_5	0xFFDA000C035200470042000 3F00	0	3e-3

We also searched these keywords within JPEG and non-JPEG encoded data chunks to measure the false alarm rate of each keyword on identifying the presence of start of the scan point of a JPEG fragment. Our results show that none of these keyword patterns are encountered within JPEG encoded data while only 0.1% (202 out of 198K) of non-JPEG encoded data chunks containing at least one of these keywords. Thus, it is possible to assume these keywords are largely unique to JPEG coding and can be reliably used to detect the presence of partial JPEG header in a data segment.

C. Identifying JPEG Encoded Data Segments

The ability to reliably identify JPEG encoded data is very crucial for the usability of orphaned JPEG file fragment carver. In the lack of this capability, the carving technique must be run on all unidentified data blocks, making the approach impractical when dealing with large storage volumes. In fact, file and data type classification has long been a focus of research. The majority of work in this domain formulated the problem in a multi-class classification setting in which large number of features are used in conjunction with a variety of classification methods to distinguish between different file types. In these studies, JPEG has always been included as one of the file types due to its widespread adoption in still image coding. The accuracy of distinguishing JPEG encoded data from other data types is reported to vary between 87% to 98% [28]–[31].

Most notably, Karresand *et al.* [16] built a JPEG specific keyword search tool named OSCAR. Their approach is based on the premise that within encoded segment of JPEG files 0xFF byte is only allowed to be followed by a particular group of bytes including 0x00 bytes, restart (RST) markers (0xD0 to 0xD7), and end-of-image (EOI) marker (0xD9). They also imposed the constraints that RST markers will loop through the defined range of values, that 0xFF00 should be encountered at least once, and that the EOI marker should not be observed more than once. The authors tested OSCAR on 53 files from 49 distinct file types by splitting them into 4 KiB chunks (on a total of 476 JPEG and $17,132$ non-JPEG chunks). Test results obtained on this dataset revealed that the method can correctly identify 97.9% of JPEG encoded data fragments while yielding a false positive rate of 0.01%. Aside from the limited size of the dataset, one potential deficiency of this approach is due to our earlier finding that RST markers only appear in around 12% of images [14]. In addition, as we will be dealing with partial data EOI marker will not be present in most cases. This only leaves the occurrence frequency of 0xFF00 byte pattern as a useful indicator. Therefore, in practice, the OSCAR method will not be able to perform as well as expected.

In this work, we improve on the OSCAR method in two aspects. First, to increase the identification accuracy for images without RST markers, we incorporate our method of identifying stuffed bytes (or byte boundary detection [14]) used as part of the orphaned file fragment carver. Second, performance of the resulting JPEG file type identifier is validated more thoroughly on datasets that better reflect real world scenarios. More specifically, our non-JPEG dataset contains 11 times more data chunks from 20 times more file types with an overall number of data chunks that is 420 times more than that of the test setup included for the OSCAR method.

In essence, JPEG format allows only RST markers (0xFFD0 to 0xFFD7) to appear within the encoded data segment as utilized by the OSCAR method. More importantly, these markers need to appear only at the byte boundary of a coded data segment. We initially utilized this characteristic to determine the byte boundary and remove byte stuffing for successful decoding. This characteristic, however, can also be used to identify JPEG encoded data without relying on RST markers, because at the byte boundary only 8 byte patterns are allowed and 247 patterns (including 0xFF01 to 0xFFCF and 0xFFD8 to 0xFFFF except 0xFFD9) are disallowed. Hence, any occurrence of these marker patterns in the encoded bit sequence must be at a non-byte boundary location. Hence, we eliminate such position indices out of 8 possible ones and determine the count of remaining possible byte boundaries which should be exactly one for a JPEG encoded data fragment. We refer to this as *byte boundary metric (bbm)* and incorporate it with the approach of the OSCAR method to build a JPEG encoded data classifier to minimize false rejections.

For a given data segment, the ensemble method includes the following steps:

1) If the number of remaining byte boundaries is zero, the data segment is accepted as non-JPEG encoded data.
2) If the number of remaining byte boundaries is one, it is accepted as JPEG encoded data.
3) If the number of remaining byte boundaries is more than one, OSCAR is utilized to accept or reject the encoded data. That is, we check the frequency of predetermined byte patterns, as proposed by OSCAR, at all remaining byte boundaries. Then, we discard any candidate byte boundary giving a frequency below a certain threshold value. Then, we keep all remaining byte boundary candidates for further processes. The data segment is rejected if we are out of candidate byte boundaries at the end of this process.

Due to sequential nature of the decision process, this method is expected to provide more accurate classification. The false rejection (FRR) and false acceptance (FAR) rates of the method can be approximated as

$$\text{FRR} = \text{FRR}_{bbm} \times \text{FRR}_{oscar}, \quad (1)$$

$$\text{FAR} = \text{FAR}_{bbm} + \text{TRR}_{bbm} \times \text{FAR}_{oscar}, \quad (2)$$

where TRR_{bbm} is the true rejection rate of byte boundary metric, (FRR_{bbm}, FAR_{bbm}) stand for error rates of byte boundary metric, and (FRR_{oscar}, FAR_{oscar}) are corresponding rates for the OSCAR method.

To demonstrate the effectiveness of the byte boundary metric and the ensemble method, we tested both approaches and the OSCAR method on the JPEG and non-JPEG file sample datasets as summarized in Table III. In Fig. 4, we also provide empirical probability distribution functions of the number of 0xFF00 bytes encountered in both datasets in order to determine a suitable threshold value for the occurrence frequency of 0xFF00 valued byte. Our results show that using byte boundary metric alone yields error rates of (FRR=3.51%, FAR=1.21%). In [16], the OSCAR method is reported to achieve (FRR=2.10%, FAR=0.01%) error rates. However on our datasets, with the same threshold value, OSCAR performed quite poorly, achieving (FRR=69.57%, FAR=14.14%) reflecting the diversity of our non-JPEG file samples dataset. The ensemble method greatly decreased these error rates to (FRR=1.05%, FAR=3.33%) at the same threshold value. We also explored the best performing threshold (in favor of FRR) for the OSCAR method and found that setting a threshold of 2 for the 0xFF00 byte count. This yields the best performance

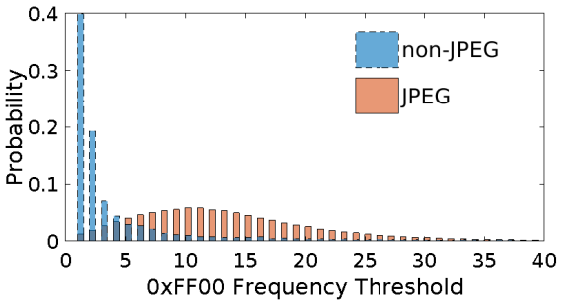


Fig. 4. Probability distribution function of the 0xFF00 byte frequency observed in JPEG (solid brown) and non-JPEG (dashed blue) data chunks.

TABLE III. ERROR RATES OF DIFFERENT APPROACHES ON DETECTING 4 KIB OF JPEG ENCODED DATA CHUNKS.

Method	Dataset	0xFF00 thr.	FRR (%)	FAR (%)
Byte-boundary	Our	-	3.51	1.21
OSCAR	[16]	9.2	2.10	0.01
OSCAR	Our	9.2	69.6	14.1
Ensemble	Our	9.2	1.05	3.33
OSCAR	Our	2	10.1	17.7
Ensemble	**Our**	**2**	**0.90**	**3.65**

by reducing error rates to (FRR=10.12%, FAR=17.72%) and (FRR=0.9%, FAR=3.65%), respectively, for the OSCAR and our ensemble methods.

Overall, the ensemble method splits a data segment of unknown type into 4 KiB data chunks, discards all non-JPEG ones, and passes all validated chunks to the orphaned file carver. If non of the 4 KiB chunks within the data segment is validated to be JPEG encoded data, the whole data segment is skipped and the next available unrecovered data segment is processed.

D. Detecting Encoding Settings and Fragmentation Points

During decoding two types of errors are encountered, namely, *invalid codewords* and *coefficient overflows*. Our carving method relies on these errors to determine the correct chroma sampling scheme and to identify a mismatch between the assumed Huffman code tables and the actual ones [14]. In this context, an invalid codeword indicates a parsing error that produces a disallowed Huffman codeword (typically an all-ones codeword in the maximum codeword length), and a coefficient overflow is observed when the number of decoded DCT coefficients in an 8×8 block exceeds 64. It must be noted that these errors may be encountered before the decoder attains synchronization with the coded bitstream as well as when Huffman code tables are selected incorrectly. Fortunately, synchronization occurs very fast (on average, after decoding 125 bytes), and the two cases can be distinguished from each other.

To detect correct decoding settings, our approach incorporates a brute-force search by trying all three chroma sampling schemes for each of the commonly used Huffman code tables and those obtained from images recovered from the same storage volume. It must be noted that end of a file fragment (EOF) can be reached anywhere in the unrecovered data segment; therefore, encoded data is processed in 1 KiB chunks. For each chunk, if the number of observed errors is above a threshold for all sampling schemes, decoding restarts with the next set of Huffman code tables. If all combinations of sampling schemes and sets of Huffman tables yield errors, that data chunk is deemed unrecoverable. Otherwise, it is accepted and the recovery continues with the next chunk using settings from the previously decoded chunk.

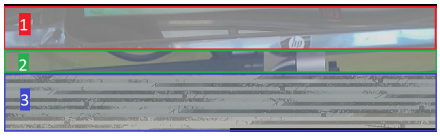


Fig. 5. Undetected fragmentation points between consecutive fragments of different images that are all coded using the same Huffman codes and the chroma subsampling scheme. Second fragment has the same width as the first one, but third one has a different width.

To determine accuracy in identifying decoding settings, we conducted tests on 7HT dataset which contains data chunks of size 1 KiB encoded using different sets of Huffman code tables. Each data chunk is decoded assuming every combination of the three chroma sampling schemes and seven sets of Huffman code tables. The decoding setting that yielded the minimum number of invalid codewords and coefficient overflows is considered to be the correct one, and the identified sampling scheme and Huffman tables are compared to the actual ones to determine the accuracy. Results show that for 1 KiB data chunks, setting error thresholds to 2 invalid codewords and to 5 coefficient overflows yield the most accurate results, (FRR=0.29%, FAR=0.35%), in identifying the correct setting. These results overall show that by working on smaller chunks of JPEG encoded data we can detect fragmentation points in the unrecovered data segments even if the EOI marker is not encountered and effectively determine the correct set of Huffman tables through number of decoding errors.

A point worth emphasizing is that when a given data segment includes fragments from multiple files laid consecutively and encoded under the same settings (*i.e.*, the same Huffman code tables and the subsampling scheme), Jpg*Scraper* won't be able to detect a fragmentation point between these fragments. This will result with rendition of a composite image as shown in Fig. 5. Although this type of an error cannot be detected during decoding, the resulting image will have visually meaningful content and further analysis can be pursued at the spatial domain [12].

IV. IMPACT ON PRACTICE

In this section, we attempt to measure the impact of orphaned JPEG file fragment carving on the practice of digital forensics. To quantify the amount of new evidence that can be recovered in the form of orphaned file fragments, we acquired a set of used memory cards. These cards are then subjected to data recovery by state-of-the-art photo carving tools to identify data segments that escaped from recovery. These unrecovered

data segments are then carved by Jpg*Scraper* to determine its effectiveness.

A. Data Collection

To determine the utility of our carver in recovering data, we acquired a collection of used secure digital (SD) memory cards from *ebay.com*. These external storage drives are used by many devices, such as smartphones, tablets, and digital cameras, to store additional data. Since multimedia data typically requires large storage space and is hard to transfer from device to device, our premise was that a used memory card will have some JPEG files stored in them. Cards were bought in small quantities from three different vendors and included 1, 2,16-GiB micro SD cards and 16-GiB SD cards in the quantities of 2, 50, 25, and 10, respectively. Our initial analysis verified that all of 2-GiB micro SD cards and 16-GiB SD cards were formatted properly by overwriting their contents with all zeros and that two of the 16-GiB micro SD cards were faulty. Therefore we could not extract any data from these cards. The rest of the cards were only quick formatted by creating a new file system without writing over the existing content. By obtaining forensic images of 25 micro SD cards, of sizes 1 GiB and 16 GiB, we obtained a total of 342.13 GiB of data to perform tests.

B. Carving Results

Table IV presents step-by-step carving results for each SD card through the introduced carving pipeline including identification and recovery of JPEG fragments with full, partial, and no header information.

Recovering JPEGs with complete headers: Using two state-of-the-art photo recovery tools, namely the Adroit Photo Forensics (APF) [21] and Blade Forensic Recovery (BFR) [32] tools, we recovered all files with valid headers. The number of recovered images by each tool, shown under the *nrof-rec* column in Table IV, demonstrate that APF performed better than BFR on all SD cards. Accordingly, APF recovered a total of 182,150 images with intact file meta-data (*full-header* master column). These included 50,196 images with thumbnails while the remaining were only thumbnails without the corresponding high-resolution images. These images collectively occupied 9.83 GiB of space, accounting for 2.87% of all memory card data. Size of recovered data for each SD card is presented under the *size-rec* column.

In our initial analysis, recovered images were found to be captured by 153 camera models from 25 manufacturers. Further analysis revealed that 3,145 of the JPEG files have missing EOI markers, indicating that those images were only partially recovered. As expected a great majority of the images were stored continuously with no gaps left between file blocks, with 5,388 (11.42%) images stored in a fragmented manner. This relatively high amount of fragmentation found in comparison to other studies [5], [33] indicate that analyzed memory cards have been used heavily for storage.

Extracting header statistics: We then collated encoding parameters from recovered JPEG files, on a per-card basis, to exploit spatial locality of data when performing carving. This data also helped us in assessing the validity of parameters obtained from the Flickr dataset on an independent dataset.

Our examination of the recovered images revealed that 98.45% of the images were encoded in JPEG baseline mode. A total of 8,558 distinct sets of Huffman tables were identified, of which 8,220 (96.05%) were used only once for encoding 17.42% of the images. Moreover, the recommended set of Huffman code tables (HT-1) was used in encoding of 78.85% of the recovered images while HT-2 and HT-3 were, respectively, used in 0.13% and 0.12% of the images. All four chroma subsampling schemes were encountered but with a different order of precedence than our earlier observation (with horizontal and vertical sampling deployed in 80.2%, horizontal sampling in 10.77%, no-sampling in 8.6%, and vertical sampling in 0.06% of all images). Out of the 3,635 distinct image dimensions observed, 2,078 (57.17%) were unique, and resolutions varied between 5×9 pixels to 6713×5056 pixels. Finally, images were compressed using 389 distinct quantization tables, 132 of which overlapped with the Flickr dataset, at an average quality factor of 79.

Overall, these findings indicate that the default decoding parameters built into Jpg*Scraper* are comprehensive enough and even without incorporating parameters from recovered images, it can recover significant amount of orphaned fragments.

Recovering JPEGs with partial headers: Following the recovery of valid images with full headers and extraction of ExIF parameters, candidate data segments for recovery are identified by first eliminating sequential data blocks containing more than 1 KiB of all-ones and all-zeros. This left us with 44.87 GiB of combined unrecovered data in all data segments obtained from the 25 memory cards as listed under the *cand-rec* column in Table IV. It must be noted that at this stage of recovery, these data segments might include any type of file data.

In order to identify partial JPEG headers, we searched the keywords derived from SOS headers (as discussed in Sec.III-B) over *cand-rec* segments. This search yielded 77,275 data segments containing candidate partial JPEG headers. Jpg*Scraper* was able to successfully recover partial images from 17,276 of these segments with a combined size of 700 MiB. The majority of these data segments, however, included very small-sized fragments (around 2 KiB, which is smaller than the typical 4KiB required by our tool to successfully render a partial image). For each SD card, the number of candidate fragments (*nrof-cand* column) as well as the size and number (*size-rec* and *nrof-rec* columns) of recovered fragments with partial headers are listed under the *par-header* master column in Table IV. Samples of recovered images with partial metadata are presented in Fig. 6. It is worth noting that existing recovery tools will completely miss this type of images due to incomplete ExIF header even though the entropy coded data segment is intact.

Recovering JPEGs without headers: Removing all data segments associated with partial headers from the combined pool of candidate data segments (*i.e.*, *cand-rec*) reduced the pool size to 44.12 GiB. The amount of residual data that potentially includes orphaned JPEG file fragments is listed under the *cand-jpg* column of Table IV on a per-card basis.

TABLE IV. RECOVERY RESULTS OF BLADE (BFR), ADROIT (APF) AND JPG*Scraper* FROM ACQUIRED SD CARDS

SD#	size of SD	BFR	full-header (APF)		size of	par-header			size of	no-header		
		nrof-rec	nrof-rec	size-rec	cand-rec	nrof-cand	nrof-rec	size-rec	cand-jpg	size-cand	nrof-rec	size-rec
1	14984	1842	1856	140.61	450.11	98	40	6.03	444.00	32.21	851	28.54
2	15190	6875	7361	935.25	6717.45	1537	363	35.02	6680.33	483.40	6099	458.38
3	15163	1798	1845	81.15	626.88	324	122	19.21	607.31	74.82	1362	68.21
4	15091	15609	16065	121.53	7224.46	8007	1574	20.23	7193.82	57.20	3396	39.32
5	15085	4834	4901	2084.44	783.41	893	183	5.79	776.44	116.40	1112	112.33
6	15186	6257	6294	595.13	340.42	1560	422	30.26	308.21	62.37	2533	49.97
7	15163	6294	6548	659.72	4376.46	2069	521	85.38	4288.10	141.08	3399	123.32
8	15088	18063	18064	929.05	642.42	7640	1890	75.01	557.70	99.52	2439	87.51
9	15186	38	39	5.65	17.63	14	12	0.77	16.86	1.83	31	1.80
10	15082	2810	2879	682.20	863.57	800	197	20.65	841.91	23.65	197	22.82
11	15186	54	81	0.01	3864.24	54	27	0.89	3863.30	14.71	1535	7.05
12	15186	41	41	5.01	3.34	19	10	0.21	3.11	0.00	1	0.00
13	15088	68474	68575	567.36	1552.89	36076	7451	150.54	1355.25	108.07	5196	83.34
14	15186	20	20	2.57	2.34	4	2	0.12	2.22	0.93	11	0.88
15	15082	13165	13265	134.56	683.46	7189	1923	94.52	580.31	129.08	4873	109.04
16	15186	56	56	7.00	0.30	25	13	0.20	0.07	0.03	1	0.03
17	15186	377	419	225.76	1052.73	1	1	1.42	1051.32	1.40	30	1.20
18	15186	16369	16463	987.15	1298.53	6475	1296	55.40	1234.88	267.00	3738	255.00
19	15190	3552	5097	978.03	8662.82	1310	407	44.75	8616.43	254.57	7599	184.29
20	15186	497	770	11.17	5528.72	240	69	0.83	5527.60	18.13	1677	10.51
21	15186	1828	2071	112.84	195.11	404	114	3.13	191.47	17.59	619	14.59
22	15186	8044	8101	721.41	934.91	2163	533	17.34	914.74	98.18	6297	75.24
23	15186	928	1067	53.15	111.17	371	105	2.22	108.51	13.69	364	12.48
24	940	176	177	12.80	9.70	1	0	0.00	9.70	1.30	6	1.29
25	967	95	95	7.02	7.24	1	1	0.07	7.17	0.65	21	0.60
Total	350339	178096	182150	10060.58	45950.32	77275	17276	669.98	45180.74	2017.81	53387	1747.74

Sizes are in MiB.



Fig. 6. Sample recovered images with partial ExIF headers.

To identify JPEG file fragments, we applied the ensemble classifier introduced in Sec.III-C to *cand-jpg* data segments which led to identification of 1.97 GiB of JPEG encoded data. Then each candidate JPEG encoded data bearing segment is carved, and the resulting partial image is validated to exhibit natural image statistics [26]. This process resulted with 53,387 partial images recovered from 1.71 GiB of data comprising orphaned JPEG file fragments. The *no-header* master column of Table IV provides for each SD card the total size of JPEG encoded data chunks (*size-cand* column) along with the total number and the size of data segments that Jpg*Scraper* was able to recover partial images (*nrof-rec* and *size-rec* columns) successfully.

C. Overall Evaluation

Figure 7 summarizes our overall findings on the data obtained from used SD cards. According to our analysis, only 16% of total SD card storage was occupied with some type of data, of which 22% was determined to be in JPEG encoding format. Actual recovery results show that state-of-the-art tools (*i.e.*, APF and BFR) was at best able to recover 81% of the JPEG encoded data. Hence, 19% of the stored JPEG file data (24% of recovered JPEG files by APF) was left beyond the reach of analysts. Jpg*Scraper* fills this gap by recovering the remaining JPEG encoded data for which associated encoding metadata may be missing or only partially available.

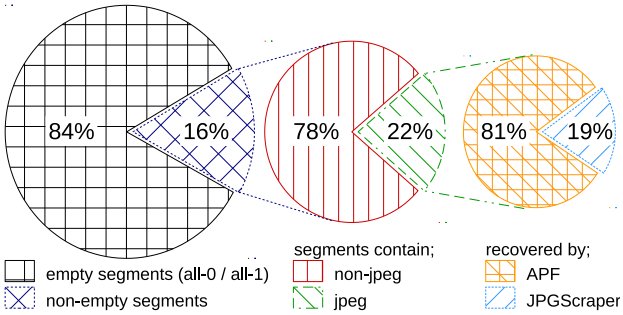


Fig. 7. Summary of recovery results from the SD cards.

V. COMPARISON WITH EXISTING WORKS

In this section, we provide a comparative analysis of both end-to-end recovery and JPEG data type identification capabilities using the DFRWS-2007 dataset since it is the only common dataset used by existing works. This challenge disk contains 331 MiB of file data including 67 complete non-JPEG and 18 complete JPEG files. It also contains JPEG files embedded in other files and JPEG thumbnails.

A. Recovery Capability

We run Jpg*Scraper* on residual segments after excluding recovered ones by using APF, JPGCarve, rx_myKarve, and PattrecCarv tools. Before running our tool, we provided it with recovered image headers. Then, we measured the percentage of additional JPEG file data that could be recovered by Jpg*Scraper*. Recovery results for all methods and additional amount of JPEG file data that can be recovered by Jpg*Scraper*

are presented in Table V along with their execution times. When determining the improvement due to Jpg*Scraper* as compared to other methods that report results in the number of recovered images (without specifying them), we assumed those images to have the largest size in the dataset, thereby upper bounding the amount of data each method recovered.

TABLE V. COMPARING JPG*Scraper* WITH APF, JPGCARVE, RX_MYKARVE AND PATTRECCARV METHODS ON DFRWS-2007

Method	original full	original inc.	embedded full	embedded inc.	time (sec.)	additional JPEG data carved by Jpg*Scraper*
Jpg*Scraper*	-	-	-	-	362	-
APF	13	5	173	10	67.32	0.12%
JPGCarve	18	-	-	-	28.50	11.91%
rx_myKarve	3	14	17	-	-	∼60%
PattrecCarv	4	-	26	-	27	∼56%

Adroit Photo Forensic: APF recovered 13 of JPEG files fully and five of them partially. It also recovered 173 full and 10 incomplete embedded JPEG files or thumbnails. Even though the challenge is not intended for recovering orphaned file fragments, we were able to recover additional fragments of incompletely carved embedded JPEG fragments as presented in Fig. 8. Overall, we recovered JPEG files from the 9.09% of the whole dataset in which 0.12% of all JPEG data is recovered additionally by Jpg*Scraper*. To the best of our knowledge, this is the most recovered amount (9.09%) of JPEG file data from the DFRWS-2007 dataset. Thus, we considered this amount as the ground-truth for the comparison of the following tools.

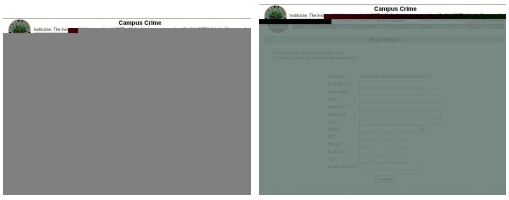


Fig. 8. Incompletely recovered JPEG file by APF (left) and its recovered version by Jpg*Scraper* (right).

JPGCarve: This is an advanced carving tool that includes many sub-processes to estimate cluster size, recover single-fragment JPEGs, discard clusters without encoded JPEG data (called Reduce method), and carve multi-fragment JPEGs [12]. JPGCarve reported that only 5.9% of the whole disk contains single- and multi-fragment JPEG files. This relatively low performance is mainly due to not recovering embedded JPEG files and JPEG thumbnails. Although JPGCarve was reported to outperform APF on different datasets, both tools recovered the same amount of multi-fragment JPEG files on the DFRWS-2007 challenge disk. In comparison, Jpg*Scraper* recovered 11.91% of all JPEG data additionally in the form of thumbnails and embedded JPEG files.

rx_myKarve: This tool is built on top of x_myKarve and myKarve [8]–[10]. These works handle single-fragment and linearly fragmented JPEG files and thumbnails. Since rx_myKarve is the most advanced version of them with multiple sub-processes (*e.g.*, discriminating JPEG encoded data with ELM technique [17] and reassembling the recovered fragments) and has an evaluation on the DFRWS-2007 challenge, we used it for comparison. It is reported that rx_myKarve successfully recovered all related JPEG files (three full JPEG files and 17 thumbnails) and 14 incomplete JPEG files additionally. Nevertheless, most part of JPEG file data embedded within other files were not recovered. Again, even if we assume all recovered fragments by rx_myKarve have the biggest size, Jpg*Scraper* could recover at least 60% of all JPEG files additionally.

TABLE VI. COMPARING OUR ENSEMBLE MODEL WITH REDUCE, ELM, AND OSCAR METHODS ON DFRWS-2007 CHALLENGE

Method	% of disk contain non-JPEG	% of disk contain JPEG	Identification Accuracy(%)	time (sec.)
Ground-truth	90.91	9.09	-	-
Ensemble/Jpg*Scraper*	89.99	10.01	99.08	8.65
Reduce/JPGCarve	82.4	5.90	[85-88]	2.13
ELM/rx_myKarve	-	-	90.15	-
OSCAR	94.7	5.53	93.69	43

PattrecCarv: This tool only focuses on recovering thumbnails and embedded JPEG files by utilizing unique hex patterns [22]. PattrecCarv is reported to recover 26 thumbnails or embedded JPEG files successfully, 4 additional JPEG files and 3 non-JPEG data files as thumbnail. Assuming, again, all recovered files are the biggest ones, Jpg*Scraper* could recover around 56% of additional JPEG files.

Overall, results presented in Table V show that Jpg*Scraper* can recover less additional data when coupled with APF by spending additional six minutes. However, it can increase other cavers' performances by recovering additional thumbnails, embedded JPEGs, or orphaned file fragments related to incompletely carved original images.

B. JPEG Data Identification Capability

To compare our method of discriminating JPEG encoded data with those used by other carving tools we performed tests on the DFRWS-2007 image disk. As recovery results revealed, 9.09% of data stored on the challenge disk is JPEG data. Correspondingly, at the most, 90.91% of disk data can be non-JPEG. Assuming this as the ground truth, we compared our method, the OSCAR, the Reduce method used by JPGCarve, and the ELM method used by rx_myKarve tools in correctly identifying JPEG data type on the DFRWS-2007 image disk. Results presented in the Table VI show that our method outperforms all other methods with an accuracy of 99.08% with a relatively short execution time. Therefore, we can conclude that our data type identification method can very precisely constrain the search space for JPEG file fragments, thereby increase the carving performance and decrease overall execution time of all carving methods. Results also show that our byte-boundary technique reduced the execution time of file type identification while coupling with OSCAR method.

VI. CONCLUSIONS

This work introduces a full fledged orphaned JPEG file fragment carving tool, called Jpg*Scraper*, by addressing all the challenges related to practical deployment. It also demonstrates the potential of the tool in uncovering new evidence that

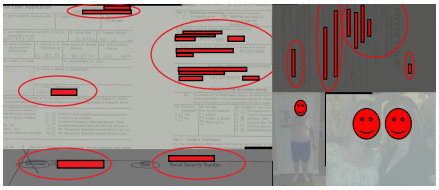


Fig. 9. Recovered partial images with sensitive personal information.

cannot be captured by existing tools on used SD cards. Even though Jpg*Scraper* recovered smaller amount of image data as compared to images with complete headers (19% vs 81%), this new class of evidence is important in two main aspects. First, orphaned file fragments are typically associated with images deleted days or months before those with complete headers. Therefore, it provides analysts with evidence of events that happened further in the past. Second, since in an investigative setting suspects will tend to delete images of criminal nature more frequently, the value of extracted evidence might have higher probative value.

Findings on used SD cards also have significant implications on user privacy as it recovers data that were actually deleted by users. Our manual inspection of the recovered fragments from SD cards demonstrate that it is possible to obtain a broad range of personal information, including financial documents, communication records, personal details, and highly sensitive photos of the former owners of the cards. Fig. 9 presents a sample collection of such photos recovered from orphaned fragments.

REFERENCES

[1] R. R. Ali, K. Mohamad, S. Jamel, and S. K. A. Khalid, "A review ofdigital forensics methods for jpeg file carving." *Journal of Theoretical and Applied Information Technology*, vol. 96, no. 17, 2018.

[2] G. G. Richard III and V. Roussev, "Scalpel: A frugal, high performance file carver." *in Proc. of Digital Forensic Research Workshop*, 2005.

[3] B. Carrier. (2015, March 10) The sleuth kit and autopsy: forensics tools for linux and other unixes. [Online]. Available: www.sleuthkit.org

[4] N. Memon and A. Pal, "Automated reassembly of file fragmented images using greedy algorithms," *IEEE Transactions on Image Processing*, vol. 15, no. 2, pp. 385–393, 2006.

[5] S. L. Garfinkel, "Carving contiguous and fragmented files with fast object validation," *in Proc. of Digital Investigation*, vol. 4, pp. 2–12, 2007.

[6] A. Pal, H. T. Sencar, and N. Memon, "Detecting file fragmentation point using sequential hypothesis testing," *in Proc. of Digital Investigation*, vol. 5, pp. 2–13, 2008.

[7] M. I. Cohen, "Advanced jpeg carving," in *Proceedings of the 1st international conference on Forensic applications and techniques in telecommunications, information, and multimedia and workshop*. ICST (Institute for Computer Sciences, Social-Informatics and , 2008, p. 16.

[8] K. M. Mohamad, A. Patel, and M. M. Deris, "Carving jpeg images and thumbnails using image pattern matching," in *2011 IEEE Symposium on Computers & Informatics*. IEEE, 2011, pp. 78–83.

[9] N. A. Abdullah, K. M. Mohamad, R. Ibrahim, and M. M. Deris, "X_mykarve: Non-contiguous jpeg file carver," *International Journal of Digital Crime and Forensics (IJDCF)*, vol. 8, no. 3, pp. 63–84, 2016.

[10] R. R. Ali and K. M. Mohamad, "Rx_mykarve carving framework for reassembling complex fragmentations of jpeg images," *Journal of King Saud University-Computer and Information Sciences*, 2019.

[11] N. A. Abdullah, R. Ibrahim, K. M. Mohamad, and N. A. Hamid, "Carving linearly jpeg images using unique hex patterns (uhp)," in *Proc. of the First International Conference on Advanced Data and Information Engineering*. Springer, 2014, pp. 291–300.

[12] J. De Bock and P. De Smet, "Jpgcarve: an advanced tool for automated recovery of fragmented jpeg files," *IEEE Transactions on Information Forensics and Security*, vol. 11, no. 1, pp. 19–34, 2016.

[13] H. T. Sencar and N. Memon, "Identification and recovery of JPEG files with missing fragments," *in Proc. of Digital Investigation*, vol. 6, pp. 88–98, 2009.

[14] E. Uzun and H. T. Sencar, "Carving orphaned jpeg file fragments," *IEEE Transactions on Information Forensics and Security*, vol. 10, no. 8, pp. 1549–1563, 2015.

[15] NFI. (2018, October) Defraser. [Online]. Available: sourceforge.net/projects/defraser/)www.guidancesoftware.com

[16] M. Karresand and N. Shahmehri, "Oscarfile type identification of binary data in disk clusters and ram pages," *in Proc. of Security and Privacy in Dynamic Environments*, pp. 413–424, 2006.

[17] R. R. Ali, K. M. Mohamad, S. Jamel, and S. K. A. Khalid, "Classification of jpeg files by using extreme learning machine," in *International Conference on Soft Computing and Data Mining*. Springer, 2018, pp. 33–42.

[18] E. Durmus, M. Mohanty, S. Taspinar, E. Uzun, and N. Memon, "Image carving with missing headers and missing fragments," in *IEEE Workshop on Information Forensics and Security*, 2017, pp. 1–6.

[19] E. Durmus, P. Korus, and N. Memon, "Every shred helps: Assembling evidence from orphaned jpeg fragments," *IEEE Transactions on Information Forensics and Security*, 2019.

[20] B. Carrier, E. Casey, and W. Venema. (2019, June 3) Dfrws 2007 forensics challenge overview. [Online]. Available: http://old.dfrws.org/2007/challenge/index.shtml

[21] Digital Assembly. (2019, June) Adroit photo recovery. [Online]. Available: https://adroit-photo-recovery.soft32.com/

[22] N. A. Abdullah, R. Ibrahim, and K. M. Mohamad, "Carving thumbnail/s and embedded jpeg files using image pattern matching," *Journal of Software Engineering and Applications*, vol. 6, no. 03, p. 62, 2013.

[23] E. Kee, M. K. Johnson, and H. Farid, "Digital image authentication from jpeg headers," *IEEE transactions on information forensics and security*, vol. 6, no. 3, pp. 1066–1075, 2011.

[24] C. Hass. (2019, June 4) Jpegsnoop 1.8.0 - jpeg file decoding utility. [Online]. Available: https://github.com/ImpulseAdventure/JPEGsnoop

[25] W. B. Pennebaker and J. L. Mitchell, *JPEG: Still image data compression standard*. Springer, 1993.

[26] S. Lyu, "Natural image statistics in digital image forensics," in *Digital Image Forensics*. Springer, 2013, pp. 239–256.

[27] International Telecommunication Union, "Information technology-digital compression and coding of continuous-tone still images: Requirements and guidelines," Tech. Rep. CCITT Rec. T.81, 1992.

[28] W. C. Calhoun and D. Coles, "Predicting the types of file fragments," *Digital Investigation*, vol. 5, pp. S14–S20, 2008.

[29] C. J. Veenman, "Statistical disk cluster classification for file carving," in *Information Assurance and Security, 2007. IAS 2007. Third International Symposium on*. IEEE, 2007, pp. 393–398.

[30] L. Sportiello and S. Zanero, "File block classification by support vector machine," in *Availability, Reliability and Security (ARES), 2011 Sixth International Conference on*. IEEE, 2011, pp. 307–312.

[31] Q. Li, A. Y. Ong, P. N. Suganthan, and V. L. Thing, "A novel support vector machine approach to high entropy data fragment classification." in *SAISMC*, 2010, pp. 236–247.

[32] Digital Detective Group Ltd. (2019, June) Blade forensic data recovery. [Online]. Available: http://www.bladeforensics.com/

[33] D. T. Meyer and W. J. Bolosky, "A study of practical deduplication," *ACM Transactions on Storage (TOS)*, vol. 7, no. 4, p. 14, 2012.